

CodeRAG - Local AI Code Assistant

Technical Documentation

1. What This Project Is

CodeRAG is a local Retrieval-Augmented Generation (RAG) application designed to help developers understand and navigate large codebases using natural language.

The system combines:

- Ollama
- Llama 3.2
- mxbai-embed-large
- ChromaDB
- LangChain
- Streamlit

to create a private AI-powered code assistant.

2. Objectives

The project aims to:

- Improve codebase understanding
- Reduce onboarding time for developers
- Enable semantic code search
- Maintain complete source-code privacy
- Provide explainable answers with citations

3. System Architecture

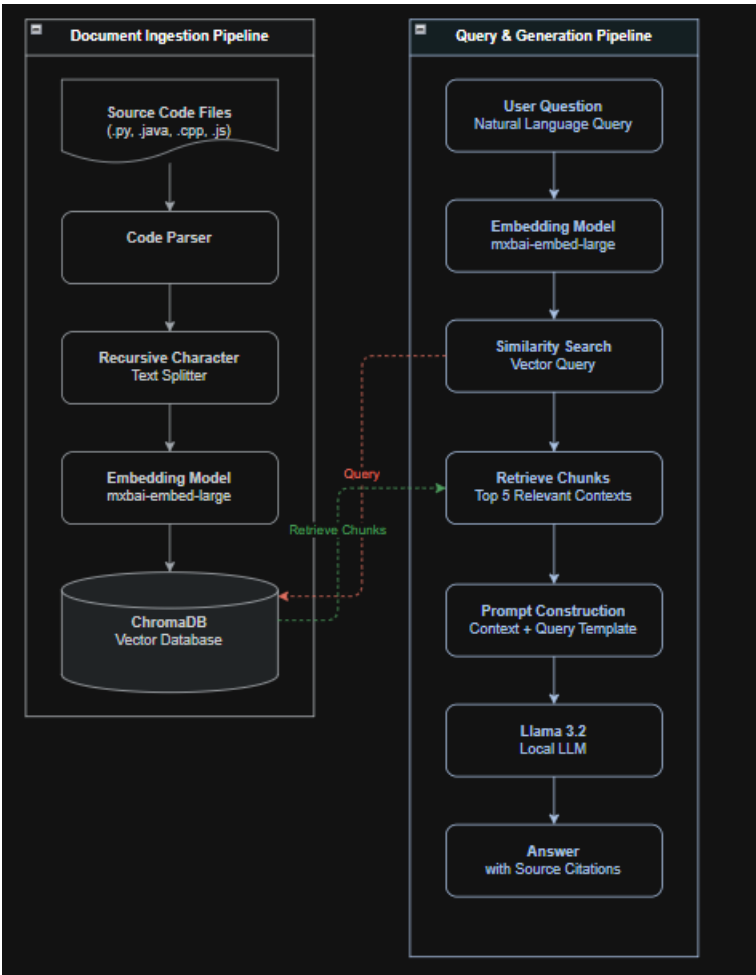


Figure 1 - System Architecture

Explanation:

Source code files are parsed, chunked, embedded, and stored in ChromaDB. User questions undergo the same embedding process and retrieve relevant code chunks before being sent to the LLM.

4. Code Ingestion Pipeline

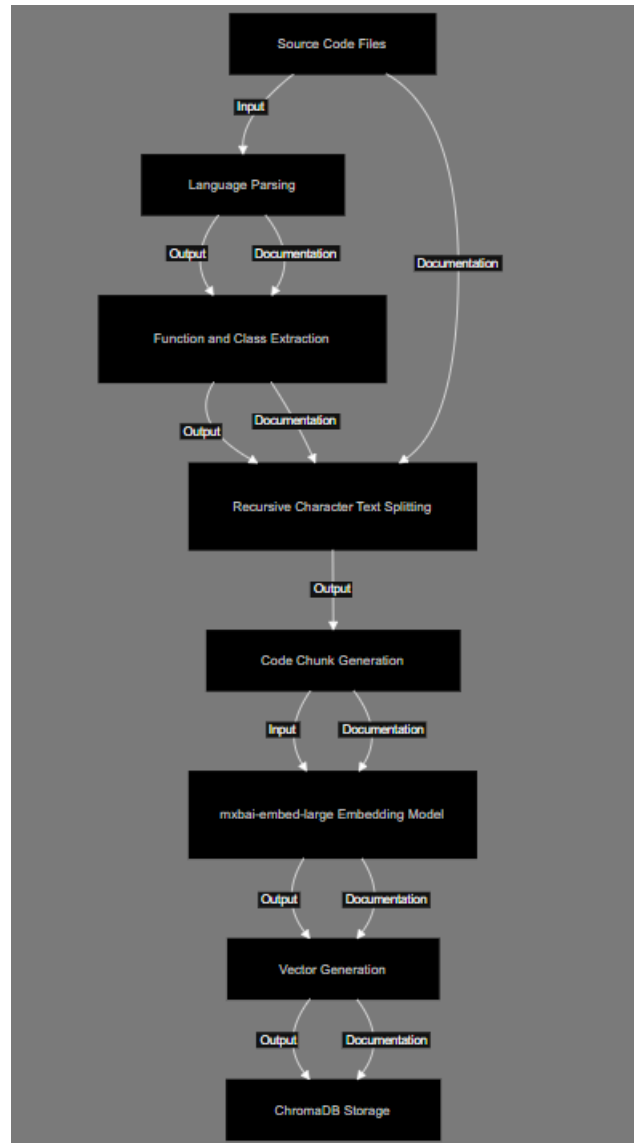


Figure 2 - Code Ingestion Pipeline

The ingestion phase converts source code into searchable vectors.

Steps:

1. Source code uploaded
2. Language parsing

3. Function and class extraction
4. Recursive text splitting
5. Embedding generation
6. ChromaDB storage

5. Query Processing Workflow

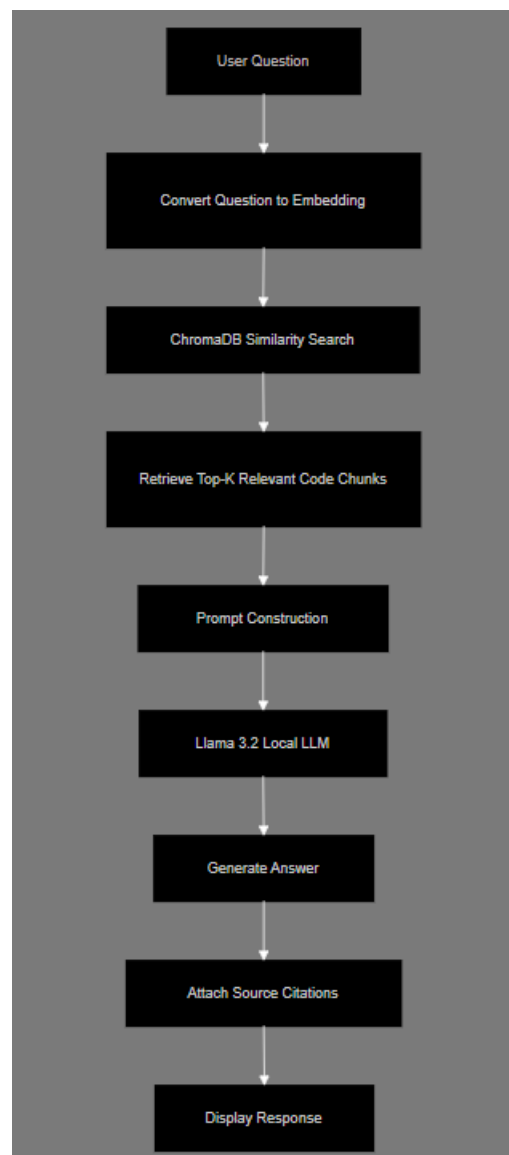


Figure 3 - Query Processing Workflow

When a user asks a question:

1. Question converted into embedding
2. ChromaDB performs similarity search
3. Top-K relevant chunks retrieved
4. Prompt constructed
5. Llama 3.2 generates a response
6. Citations returned

6. ChromaDB Collection Schema

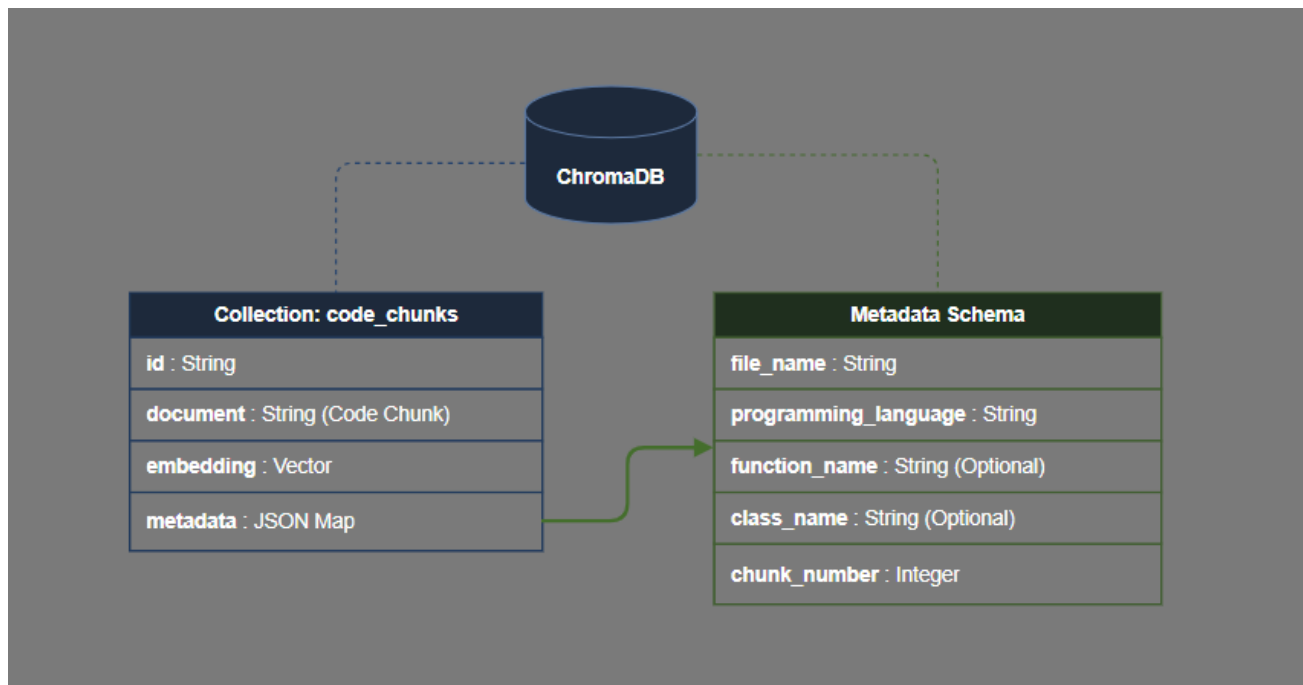


Figure 4 - ChromaDB Collection Schema

Each collection stores:

- Original code text
- Embedding vectors
- Metadata

Metadata includes:

- File name
- Programming language
- Function name

- Class name
- Chunk number

This enables traceable source citations.

7. Traditional Search vs RAG Search

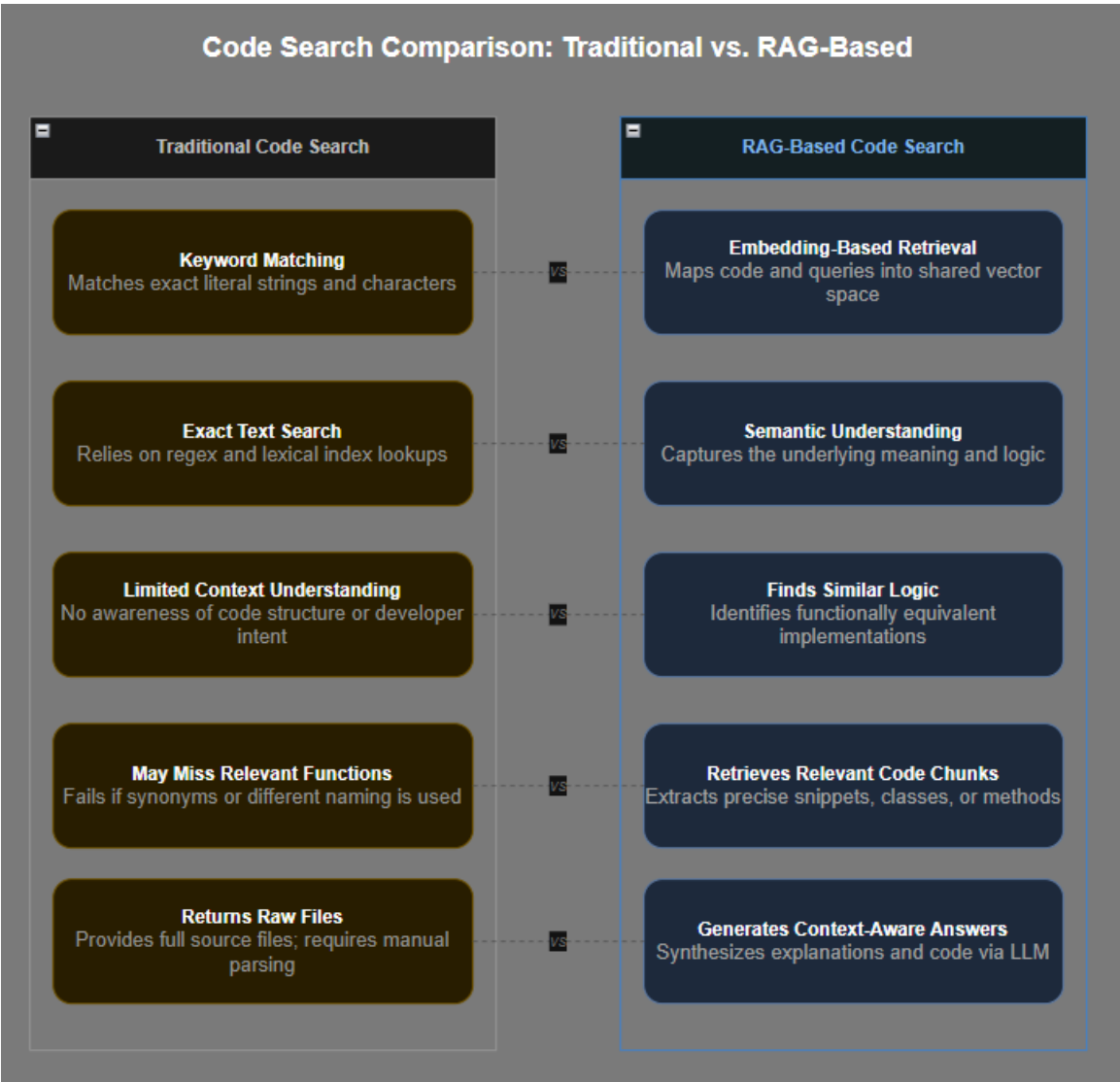


Figure 5 - Traditional Search vs RAG Search

Traditional Search:

- Keyword matching
- Exact word dependency

- Limited context awareness

RAG Search:

- Semantic retrieval
- Understands meaning
- Retrieves contextually relevant code
- Better developer experience

8. Technology Stack

Technology and its Purpose

1. Python is used for the backend logic
2. Streamlit for the User Interface
3. LangChain RAG orchestration
4. Ollama Local inference
5. Llama 3.2
6. mxbai-embed-large
7. ChromaDB

9. Database Design

ChromaDB collections contain:

- ID
- Document
- Embedding
- Metadata

The vector database enables semantic retrieval using similarity search instead of traditional keyword matching.

10. Why Embeddings Are Needed

Embeddings convert source code into numerical vectors.

Example:

```
def calculate_total(price, qty):  
    return price * qty
```

becomes a high-dimensional vector representation.

This allows the system to find semantically similar code even when variable names differ.

11. Why ChromaDB Is Needed

ChromaDB stores:

1. Original code chunks
2. Embeddings
3. Metadata

Benefits:

- Fast retrieval
- Persistent storage
- Source citations
- Collection management

12. Advantages of Local LLMs

1. No API cost
2. Full privacy
3. Offline capability
4. Suitable for enterprise repositories
5. Complete control over models

13. Future Scope

1. GitHub Repository ingestion
2. Multi-user support
3. IDE integration

14. Conclusion

CodeRAG demonstrates how local LLMs, embeddings, vector databases, and retrieval-augmented generation can be combined to build a secure and effective code understanding platform.

The system enables developers to query repositories using natural language while maintaining complete control over source code privacy.

Prepared By:
Arya Barsode